

Formato de archivo objeto (.o) y decisiones de codificación

Este documento describe el **formato objeto** que generan `assembler1` y `assembler2`, y las **decisiones de codificación** del encoder IA-32.

El archivo objeto es de **texto** (no binario). Se eligió texto para que sea fácil de inspeccionar y depurar a mano, que es justamente uno de los objetivos del proyecto. Toda la información (encabezado, secciones, símbolos, relocalaciones y código) vive en un solo `.o`.

1. Gramática del archivo objeto

El archivo se lee token por token con `fscanf`. Las líneas son:

```
IA32OBJ 1
SRC <nombre_fuente>
SECTION .text <tamaño>
SECTION .data <tamaño>
SECTION .bss <tamaño>
SYMBOL <nombre> <GLOBAL|LOCAL|EXTERN|CONST> <seccion|-> <valor>
...
RELOC <seccion> <offset> <simbolo> <ABS32|REL32> <addend>
...
CODE .text <n>
<n bytes en hexadecimal, 16 por línea>
CODE .data <n>
<n bytes en hexadecimal, 16 por línea>
END
```

Reglas:

- **IA32OBJ 1** : número mágico + versión. Sirve de encabezado.
- **SRC** : nombre del archivo fuente original (informativo).
- **SECTION** : una línea por sección. El tamaño está en **decimal**. La sección `.bss` declara su tamaño pero **no** lleva bytes (`CODE`), porque es

espacio reservado sin contenido.

- **SYMBOL** : tabla de símbolos. El tipo puede ser:
 - **GLOBAL** — etiqueta definida y exportada (`global`).
 - **LOCAL** — etiqueta definida, visible solo dentro del módulo.
 - **EXTERN** — símbolo usado pero definido en otro módulo (`extern`).
 - **CONST** — constante definida con `equ` (no ocupa memoria).
 - La columna sección es `.text` , `.data` , `.bss` o `-` (para const/extern).
 - El *valor* es el offset dentro de su sección (decimal). Para **CONST** es el valor literal.
- **RELOC** : una línea por relocación pendiente. Indica en qué sección y offset hay que parchar, qué símbolo se necesita, de qué tipo es la relocación y el *addend* (constante a sumar).
- **CODE** : bloque de bytes de una sección, en **hexadecimal** (mayúsculas, 16 bytes por renglón). El número `n` indica cuántos bytes siguen.
- **END** : marca el final.

Ejemplo real (módulo `mod1.asm`):

```
IA32OBJ 1
SRC mod1.asm
SECTION .text 15
SECTION .data 4
SECTION .bss 0
SYMBOL valor LOCAL .data 0
SYMBOL main GLOBAL .text 0
SYMBOL helper EXTERN - 0
RELOC .text 2 valor ABS32 0
RELOC .text 8 helper REL32 0
CODE .text 15
8B 05 00 00 00 00 50 E8 00 00 00 00 03 C1 C3
CODE .data 4
2A 00 00 00
END
```

2. Secciones y diseño en memoria

- `.text` : código.
- `.data` : datos inicializados (`db` , `dw` , `dd`).

- `.bss` : datos no inicializados (`resb` , `resw` , `resd`); solo se cuenta el tamaño.

Cada símbolo guarda un **offset relativo al inicio de su sección**. Las direcciones absolutas se calculan hasta el enlace.

3. Relocaciones

Hay dos tipos:

Tipo	Significado	Fórmula que aplica el linker
<code>ABS32</code>	dirección absoluta 32 bits	<code>campo = sym_final + addend</code>
<code>REL32</code>	desplazamiento relativo	<code>campo = sym_final + addend - (sitio + 4)</code>

Donde `sitio` es la dirección final del propio campo y el `+4` corresponde a que en IA-32 los saltos/llamadas relativos se miden desde el final de la instrucción (que ocupa 4 bytes de desplazamiento).

Qué genera una relocación y qué se resuelve en el ensamblado:

- Un salto/llamada **relativo a una etiqueta de la misma sección** se resuelve directamente en el ensamblador (queda como código posicional, sin relocación).
- Un salto/llamada **a otra sección** o **a un símbolo externo** se deja como relocación `REL32` .
- El **uso de una dirección** (p.ej. `mov eax, etiqueta` , `mov eax, [etiqueta]` , `dd etiqueta`) siempre genera una relocación `ABS32` .
- Una constante `equ` usada como valor se sustituye en sitio (sin relocación).

4. Decisiones de codificación del encoder (IA-32)

El encoder construye, según corresponda:

`[opcode]` `[ModRM]` `[SIB]` `[desplazamiento]` `[inmediato]` .

4.1. Campo de registro (3 bits)

Reg	Código	Reg	Código
-----	--------	-----	--------

Reg	Código	Reg	Código
EAX	0	ESP	4
ECX	1	EBP	5
EDX	2	ESI	6
EBX	3	EDI	7

4.2. ModRM y SIB

- Operando **registro** → `mod = 11`, `rm = código del registro`.
- Operando **memoria**:
 - Se usa **SIB** si hay registro índice, o si la base es `ESP`, o si no hay base (solo índice escalado).
 - Tamaño del desplazamiento:
 - con símbolo → `disp32 (mod = 10)`;
 - `disp == 0`, con base y base ≠ `EBP`, sin símbolo → sin desplazamiento (`mod = 00`);
 - cabe en 8 bits → `disp8 (mod = 01)`;
 - en otro caso → `disp32 (mod = 10)`.
 - Memoria directa** `[disp]` o `[símbolo]` → `mod = 00`, `rm = 101`, `disp32`.
 - `[EBP]` con desplazamiento 0 se fuerza a `disp8 = 0` (no existe `mod=00` con base `EBP`).
 - El índice **no** puede ser `ESP` (se reporta como error).
 - Las escalas válidas del SIB son **1, 2, 4, 8**.

4.3. Opcodes implementados

- `mov`: `B8+r (reg,imm32)`, `C7 /0 (r/m,imm32)`, `8B /r (reg,r/m)`, `89 /r (r/m,reg)`.
- ALU(`add 01/03`, `or 09/0B`, `and 21/23`, `sub 29/2B`, `xor 31/33`, `cmp 39/3B`) y forma inmediata `81 /digit (add0, or1, and4, sub5, xor6, cmp7) + imm32`.
- `inc FF/0`, `dec FF/1`, `not F7/2`, `neg F7/3`, `mul F7/4`, `div F7/6`, `idiv F7/7`.
- `push`: `50+r`, `68 imm32`, `FF /6`. `pop`: `58+r`, `8F /0`.
- `lea`: `8D /r` (solo fuente en memoria).

- `imul` : `F7 /5` (un operando)y `0F AF /r` (dos operandos).
 - `movzx` : `0F B6 /r` (versión simplificada, fuente de 8 bits).
 - Saltos: `jmp E9 +rel32` ; `call E8 +rel32` ; condicionales `0F 8x +rel32` (`je 84` , `jne 85` , `jg 8F` , `jge 8D` , `jl 8C` , `jle 8E`).
 - `ret C3` , `nop 90` , `int CD ib` .
-

5. Simplificaciones deliberadas (alcance del proyecto)

Para no salirnos del subconjunto pedido en el documento:

- Los **saltos siempre son cercanos** (`rel32`); no se generan formas cortas (`rel8`). Así el tamaño de cada instrucción depende solo de la forma de sus operandos, lo que hace que la pasada 1 (dos pasadas) calcule exactamente el mismo tamaño que la pasada 2.
- `movzx` e `imul` se implementan en su forma básica.
- `org` se acepta y se parsea, pero su efecto es **solo informativo** (se guarda por sección; el linker no lo usa para colocar las secciones).
- `equ` debe definirse **antes** de usarse.
- Los números pueden ser **decimales** o **hexadecimales** (`0x...`).
- Las **etiquetas requieren dos puntos** (`etiqueta:`).
- Las cadenas (`"..."`) solo se permiten en `db` y se expanden a un byte por carácter.
- La dirección base de `.text` en el linker es `0x00000000` (configurable con la constante `TEXT_BASE` en `src/minilinker.c`). `.data` y `.bss` se colocan a continuación, alineadas a 4 bytes.